A Brief Documentation of `obfusimg`

___roselle___, pigeon.hannah

February 21, 2026

Abstract

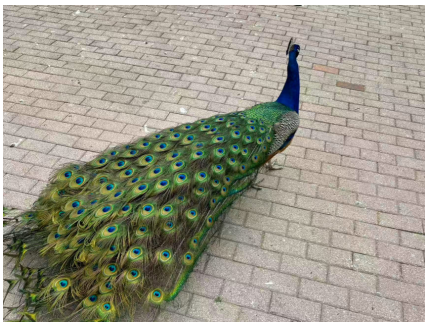
The `obfusimg` application offers multiple algorithms that obfuscates images by shuffling the pixels of the image in a deterministic and revertible way. This document serves as a semi-technical manual for the C++ implementation of the application.

1 Introduction

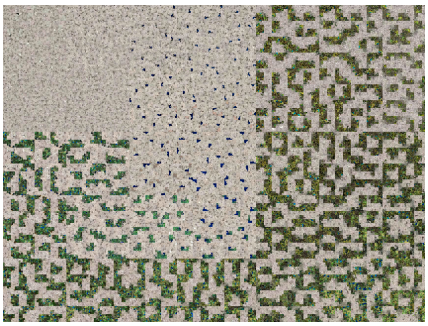
Invertible image obfuscation has a wide range of applications, from privacy-preserving image sharing (e.g. circumventing censorship) to reversible preprocessing in scientific and industrial imaging pipelines (e.g. benchmarking machine learning architectures with disrupted spatial correlation).

A relatively straightforward approach to obfuscation is by rearranging all the pixels in a given image. Each pixels is moved to a unique new position following a certain method such that the output is an image of the same dimensions (width, lengths, number of channels) but hardly recognizable (see Figure 1).

The application `obfusimg` implements various algorithms that generate this method for pixel-rearrangement. In particular, the modest computation cost associated with these algorithms allow users to run the application on any reasonable local device meaning that no information needs to be transmitted through internet prior to obfuscation.



(a) An image of a peacock (courtesy of mnrva).



(b) Peacock with all pixels rearranged.

Figure 1: The peacock in a demo image becomes unrecognizable after all the pixels are rearranged. State-of-the-art LLM's cannot reliably identify the subject of the obfuscated image (tested on ChatGPT 5.2 Thinking extended).

2 Mathematical model of an image and obfuscation

Let an image have width w px, height h px, and c channels (typically, $c = 3$ for `jpeg` files with RGB channels and $c = 4$ for `png` files with RGB and Alpha channels). We identify the image with a 1-dimensional array

$$A = \big(a_0, a_1 \ldots, a_{wh-1}\big) \tag{1}$$

where each a_i stores all c channel values of one pixel (typically, each channel value is an integer $0 \leq n \leq 255$). Here, we use row-major indexing: if a pixel has coordinates (x, y) in the original image with

$$0 \leq x < w, \qquad 0 \leq y < h, \tag{2}$$

then its array index is

$$i = i(x, y) = yw + x. \tag{3}$$

Note that this simply means we read the pixels from left to right, and top to bottom. This process is reversible as long as we know the width (or height) of the image.

An obfuscation method by rearranging all pixels can then be defined as a permutation[1]

$$\sigma\colon \{0, 1, \ldots, wh - 1\} \to \{0, 1, \ldots, wh - 1\} \tag{4}$$

and the obfuscated image is

$$A' = \big(a_{\sigma(n)}\big)_n. \tag{5}$$

In other words, the nth pixel in the obfuscated image corresponds to the $\sigma(n)^{\text{th}}$ pixel in the original image. By applying the inverse obfuscation method σ^{-1} to A', we get

$$A'' = \big(a_{\sigma^{-1}(\sigma(n))}\big)_n = \big(a_n\big)_n = A \tag{6}$$

and recover the original image.

3 Obfuscation algorithms

Each algorithm implemented in `obfusimg` is essentially a map

$$\alpha\colon \mathcal{I} \times [0, 1) \to \mathcal{P}, (I, s) \to \sigma \tag{7}$$

where $\mathcal{I}$ is the set of all images, $\mathcal{P}$ is the set of all permutations of finite sets, $I = (w, h, c, A)$ is a user-input image, $s \in [0, 1)$ is a user-defined seed, and σ is a permutation of the set $\{0, 1, \ldots, wh - 1\}$. In practice, $\alpha(I, s)$ almost never depend on c and A since c has little variability and A is prone to compression loss for `jpeg` files.

3.1 G/Hilbert curves

A Hilbert curve[2] (Gilbert curve) for a 2^n-by-2^n square is a "continuous" curve traveling via adjacent pixels through square with each pixel visited exactly once. By labeling the pixel indices with the corresponding steps, we effectively get a permutation

$$h\colon \{0, 1, \ldots, 2^{2n} - 1\} \to \{0, 1, \ldots, 2^{2n} - 1\}, \text{step} \mapsto \text{pixel index at step}. \tag{8}$$

[1] A map $f\colon A \to A$ is called a permutation of A if f is bijective. Therefore, a permutation $f\colon A \to A$ necessarily attains a unique inverse $f^{-1}\colon A \to A$.

[2] Technically, we are using finite iterations of curves leading up to Hilbert curves. Hilbert curves, like Peano curves, are space filling curves and serve as examples of *continuous* bijective maps from $[0, 1]$ to $[0, 1]^2$. This is surprising since C^1 images of lower-dimensional rectangles have zero Jordan measure.

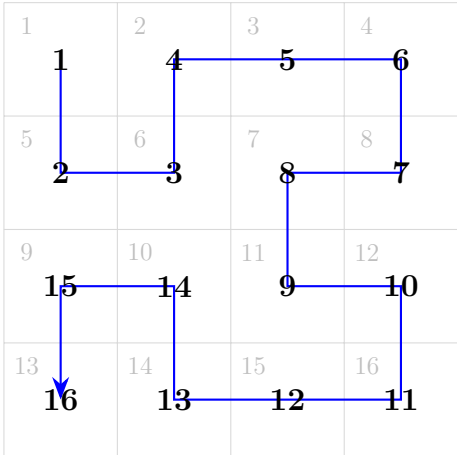


Figure 2: An example of a Gilbert curve for a 4-by-4 rectangle. The gray numerals represent the original indices and the black numerals represent the rearranged indices. In other words, $g(\text{black}) = \text{gray}$. Note that the Gilbert curve here is slightly different from the actual Gilbert curve implemented in `gilbert_curve.cpp`.

Since Hilbert curves are limits of fractals, a simple recursion gives us the Hilbert curve for an arbitrary n.

A generalized Hilbert curve (Gilbert curve) for a w-by-h rectangle is a generalized version of a Hilbert curve that is "continuous" and reaches each pixel exactly once (see Figure 2). Similarly, this curve defines get a permutation

$$g\colon \{0, 1, \ldots, wh-1\} \to \{0, 1, \ldots, wh-1\} \tag{9}$$

which we could use as our desired permutation σ.

A naïve attempt at the problem of computing Gilbert curves suggests "padding" the w-by-h rectangle to the smallest containing 2^n-by-2^n square, computing the Hilbert curve for that square, and truncating the parts of the curve outside of the rectangle. However, this is computationally inefficient as we will usually waste over half of our computation resources computing the curve outside of the rectangle. Here we use two advanced methods.

3.1.1 Compact Gilbert curves

This method[3] suggested by Chris H. Hamilton and Andrew Rau-Chaplin computes

$$w^* = \lceil \log_2(w) \rceil, \qquad h^* = \lceil \log_2(h) \rceil \tag{10}$$

as the number of bits (0 or 1) needed to represent an x and y index respectively. The method takes a point (x, y), starts with the coarsest (highest) bit level, determines the quadrant of the point in the current bit level and setting the corresponding bit(s), descends into a lower level, and repeats.

At the end of the process, one obtains an index $0 \le C(x, y) \le 2^{w^*+h^*} - 1$ that describes the position of the point on a Gilbert curve. By computing $C(x, y)$ for all $0 \le x < w$ and $0 \le y < h$, we obtain the *unnormalized* compact Gilbert function

$$C\colon \{0, 1, \ldots, wh-1\} \to \{0, 1, \ldots, 2^{w^*+h^*} - 1\}. \tag{11}$$

[3]See "Compact Hilbert indices: Space-filling curves for domains with unequal side lengths." Available at https://doi.org/10.1016/j.ipl.2007.08.034.

that is injective but usually not surjective. We normalize C by ranking the outputs to get the *normalized* compact Gilbert function

$$\tilde{g}\colon N = \{0, 1, \ldots, wh - 1\} \to N, i \mapsto \text{rank of } C(i) \text{ in } C(N). \tag{12}$$

This $\tilde{g}$ is such that $\tilde{g}(i)$ is the position of the point with index i on the "truncated curve." By taking the inverse of $\tilde{g}$, we get

$$g\colon \{0, 1, \ldots, wh - 1\} \to \{0, 1, \ldots, wh - 1\}, i \mapsto \tilde{g}^{-1}(i) \tag{13}$$

which gives the index of the ith point on the curve. This g is our desired permutation σ.

3.1.2 Exact Gilbert curves

This method[4] suggested by Jakub Červený computes a curve that travels exclusively in the w-by-h rectangle by recursively splitting a rectangle into three regions following a U-shaped path. Note that in particular cases, a diagonal move may be used. This method gives us a *normalized* exact Gilbert function

$$g\colon \{0, 1, \ldots, wh - 1\} \to \{0, 1, \ldots, wh - 1\} \tag{14}$$

which gives the index of the ith point on the curve.

Following a popular tool "小番茄图片混淆"[5], we do not use this g directly as our permutation for this case. Instead, we rearrange our pixels by shifting each pixel along the exact Gilbert curve by an offset of

$$k = \left\lfloor \frac{\sqrt{5} - 1}{2} hw \right\rceil = \left\lfloor \frac{hw}{\varphi} \right\rceil. \tag{15}$$

This means that the new pixel at index j which is originally the $g^{-1}(j)^{\text{th}}$ point on the curve should be the $(g^{-1}(j) - k)^{\text{th}}$ point on the curve after the shift. Since g is bijective, $j = g(i)$ for some i. Our desired permutation is then

$$\sigma\colon \{0, 1, \ldots, wh - 1\} \to \{0, 1, \ldots, wh - 1\}, g(i) \mapsto g(\overline{i - k}) \tag{16}$$

where $\overline{i - k}$ is understood to be mod hw. Notice that the permutation is well-defined since g is bijective.

3.2 Chaotic systems

If we simply want a highly noise-like but reproducible σ (lava lamps in Cloudflare's SF office generate very noisy data, but there is no hope of inverting the permutation afterwards), we may look at chaotic systems. very loosely[6], we characterize chaotic systems by the following properties:

[4] See https://github.com/jakubcerveny/gilbert/.

[5] See https://xfqtphx.netlify.app/.

[6] Strictly speaking, there are various rigorous mathematical definitions for chaotic systems. A most classical one is Devaney chaos defined for continuous functions $f\colon X \to X$ on a metric space (X, d). Such a function f is Devaney chaotic if it is topologically transitive, has dense periodic points, and has sensitive dependence on initial conditions. However, let us reserve that for a math course.

- The system evolves deterministically. That is, there is a well-defined function f such that $x_{n+1} = f(x_n)$ always.
- The system, with any initial state, can evolve to any region given enough time.
- The system is sensitive to initial conditions. That is, two starting points close to each other can evolve in drastically different ways.

There are many simple functions $f\colon I \subseteq \mathbb{R} \to \mathbb{R}$ that exhibit chaotic behavior. For example, the logistic map $L(x) = rx(1-x)$ for $r \lesssim 4$, the tent map $T(x) = \mu \min\{x, 1-x\}$ for $\mu \in (1, 2)$ (see Figure 3), and the doubling map $D(x) = 2x \pmod 1$ can all be used to simulate chaos.

Assume a chaotic map $f\colon I \subseteq \mathbb{R} \to \mathbb{R}$. A user-defined seed s sets the initial condition $x_0 = s$. For $0 \leq i < wh - 1$, we compute

$$x_{i+1} = f(x_i). \tag{17}$$

This gives us a noise-like array $(x_n)_n$. We may rank the x_n's by sorting, using the indices n's to break ties if necessary. This gives us the desired permutation

$$\sigma\colon \{0, 1, \ldots, wh-1\} \to \{0, 1, \ldots, wh-1\}, i \mapsto n \text{ such that } x_n \text{ has rank } i. \tag{18}$$

3.2.1 Tent map

We implement the tent map as

$$T_\mu\colon [0,1] \to [0,1], x \mapsto \begin{cases} \mu x & x < 1/2, \\ \mu(1-x) & x \geq 1/2 \end{cases} \tag{19}$$

for $0 < \mu \leq 2$. This map is highly chaotic for $\mu = 2$. However, in an actual C++ implementation multiplication by 2 is effectively a bit-wise left shift. With finite precision, we will loose all bits after approximately 53 iterations. Therefore, we set $\mu = 1.9999$.

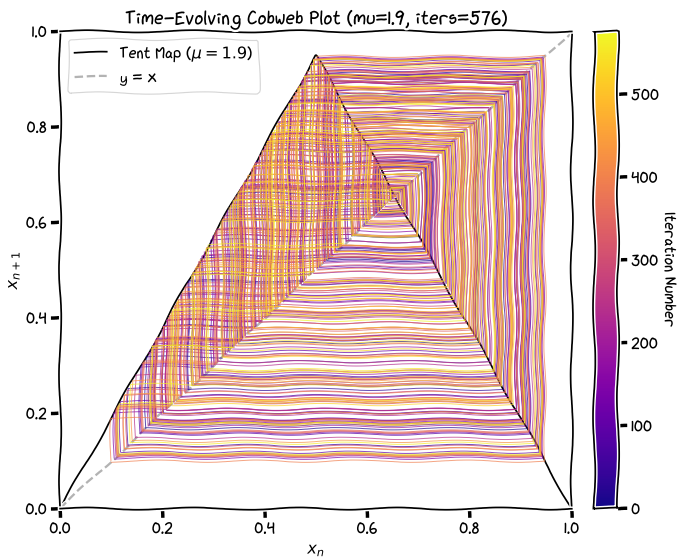


Figure 3: Evolution of the tent map with $\mu = 1.9$ and $x_0 = 0.7$ after 24^2 iterations.

Appendices

A header file for permutations module

```cpp
/// @file permutation.h
/// @author __roselle__, pigeon.hannah
/// @date 2026-02-20
/// @brief A module that handles permutations encoded as vectors of integers.
#include <vector>

/// @brief Function to normalize a given permutation by associating each
///        element with its rank in the permutation.
/// @param perm  the permutation to be normalized (std::vector<int>)
/// @return the normalized permutation (std::vector<int>)
std::vector<int> normalize_permutation(const std::vector<int>& perm);

/// @brief Function that computes a permutation by shifting a sequence a(n)
///        such that a(n) lands in the position of a(n + shift).
/// @param seq   the original sequence to be shifted (std::vector<int>)
/// @param shift the number of positions to shift the sequence (int)
/// @return the shifted sequence as a permutation (std::vector<int>)
std::vector<int> shift_sequence(const std::vector<int>& seq, int shift);

/// @brief Function to invert a given permutation.
/// @param perm  the permutation to be inverted (std::vector<int>)
/// @return the inverted permutation (std::vector<int>)
std::vector<int> invert_permutation(const std::vector<int>& perm);

/// @brief Function to apply a permutation to an image's pixel data.
/// @param pixels    the pixel data of the original image (unsigned char*)
/// @param perm      the permutation to be applied to the pixel data
///                  (std::vector<int>)
/// @param width     the width of the image (int)
/// @param height    the height of the image (int)
/// @param channels  the number of channels in the image (int)
/// @return the obfuscated pixel data (unsigned char*)
unsigned char* apply_permutation(
    unsigned char* pixels, std::vector<int> perm,
    int width, int height, int channels
);
```

B header file for Gilbert curves module

```
/// @file gilbert_curve.h
/// @author __roselle__, pigeon.hannah
/// @date 2026-02-20
/// @brief A module that computes a generalized hilbert curve for an arbitrary
///        2-dimensional rectangle.
#include <vector>

/// @brief Function to compute the nth bit of an integer in binary.
/// @param val the interger to be represented in binary (int)
/// @param bit the position of the bit to be extracted,
///        where 0 is the least significant bit (int)
/// @return the value of the nth bit (0 or 1) (int)
int get_bit(int val, int bit);

/// @brief Function to compute the sign of an integer.
/// @param x  the integer whose sign is to be computed (int)
/// @return 1 if x is positive, -1 if x is negative, and 0 if x is zero (int)
int sign(int x);

/// @brief Function to perform floor division by 2 for integers.
/// @param x  the integer to be divided (int)
/// @return the result of floor division of x by 2 (int)
int floor_div2(int x);

/// @brief Function to compute the Compact Hilbert Index for a 2D point.
///        The index can be imagined as the position of the point along a
///        Gilbert curve that fills the smallest containing rectangle of size
///        2^w_prec by 2^h_prec.
/// @param x     the horizontal coordinate of the point (int)
/// @param y     the vertical coordinate of the point (int)
/// @param w_prec the precision (number of bits) needed to represent the
///               width of the rectangle (int)
/// @param h_prec the precision (number of bits) needed to represent the
///               height of the rectangle (int)
/// @return the Compact Hilbert Index corresponding to the point (int)
/// @cite https://doi.org/10.1016/j.ipl.2007.08.034
int get_compact_index(int x, int y, int w_prec, int h_prec);

/// @brief Function to compute the compact g function for a given rectangle.
///        This g function needs to be normalized and inverted.
/// @param w  the width of the rectangle (int)
/// @param h  the height of the rectangle (int)
/// @return a vector of integers representing the $g$ function, where the value
///        at index $i$ is the Compact Hilbert Index of the point corresponding
///        to the original index $i$ in row-major order (std::vector<int>)
std::vector<int> generate_compact_g_function(int w, int h);

/// @brief Recursive function to compute the coordinates of the points along
```

```
    an
///         exact Gilbert curve by dividing the rectangle into smaller blocks.
/// @param x         the starting absolute horizontal coordinate of the
///                  current block (int)
/// @param y         the starting absolute vertical coordinate of the
///                  current block (int)
/// @param ax        the width of the major axis of the current block with
///                  signature indicating movement direction (int)
/// @param ay        the height of the major axis of the current block with
///                  signature indicating movement direction (int)
/// @param bx        the width of the minor axis of current block with
///                  signature indicating movement direction (int)
/// @param by        the height of the minor axis of current block with
///                  signature indicating movement direction (int)
/// @param img_width the total width of the image, used to compute 1D indices
///                  from 2D coordinates (int)
/// @param coordinates reference to the output vector (std::vector<int>)
/// @cite https://github.com/jakubcerveny/gilbert/
void draw_gilbert_curve(
    int x, int y, int ax, int ay, int bx, int by, int img_width,
    std::vector<int>& coordinates
);

/// @brief Function to compute the exact g function for a given rectangle.
///        This g function is normalized.
/// @param w  the width of the rectangle (int)
/// @param h  the height of the rectangle (int)
/// @return a vector of integers representing the $g$ function, where the
    value
///        at index $i$ is the row-major index of the $i$-th point in the exact
///        Gilbert curve (std::vector<int>)
std::vector<int> generate_exact_g_function(int width, int height);
```

C header file for chaotic systems module

```
/// @file chaos.h
/// @author __roselle__, pigeon.hannah
/// @date 2026-02-20
/// @brief A module that computes a chaotic permutation based on various
///        chaotic maps and an initial seed.
#include <vector>
#include <functional>

/// @brief Computes the value of the tent map at a given point.
/// @param x  the input value, typically in the range [0, 1] (double)
/// @param mu the parameter of the tent map,
///           note that mu = 2.0 loses chaos after about 53 iterations due to
///           binary representation and should be avoided (double)
/// @return the output value of the tent map (double)
double tent_map(double x, double mu);

/// @brief Generates a chaotic permutation of integers from 0 to n-1 based on
///        the a chaotic map and an initial seed.
/// @param n     the size of the permutation (int)
/// @param seed  the initial seed for the chaotic map (double)
/// @param map_func the chaotic map to be used (std::function<double(double)>)
/// @return a vector of integers representing the chaotic permutation, where
std::vector<int> generate_chaotic_permutation(
    int n, double seed, std::function<double(double)> map
);
```